

```

    val: {
      uint_val: 6243543
    }
  }
  update: {
    path: {
      elem: {
        name: "out-unicast-pkts"
      }
    }
    val: {
      uint_val: 31339
    }
  }
}
# output omitted for brevity
}

```

We've now provided an introduction to using the Python and Go libraries to communicate with modern programmatic network APIs. Now, we are going to shift gears and talk about using SSH in Python, as SSH is still the most widely deployed interface on network devices.

## The Netmiko Python Library

Using CLI commands, SSH has been the de facto way network engineers and operators manage their infrastructure. Commands are passed over a persistent SSH connection to a network device, the device interprets them, and it responds with text that is viewable by a human on a terminal window. SSH does not use structured encoded data such as XML or JSON over the wire. While SSH is not a modern or a programmatic API, it is important to have an understanding of how to use Python to automate network operations with SSH for three reasons:

- Not all devices support a programmatic API.
- You may want to automate the turning on of the API.
- Even if you're automating a device with an API:
  - It's good to have a backup plan.
  - Not all operations of a device may be supported with the API. This is not ideal, as it shows immaturity in the underlying API.

In this section, we show how to get started with a popular open source SSH client for Python called *Netmiko*. Netmiko's purpose is to simplify SSH device management specifically for network devices.



Do not underestimate the utility of the SSH Netmiko library, as it's proven useful in providing a smooth transition from traditional network CLI management to network automation, and it's heavily used and developed, as you can see in the [Netmiko GitHub Contributors page](#).

We're focused on Netmiko, as it provides a lower barrier to entry and already understands how to communicate with many network device types. Netmiko has varied support for dozens of device types, including those from Arista, Brocade, Cisco, Dell, HPE, Juniper, Palo Alto Networks, Linux, and many more; check the [documentation](#) for updated information. The great thing about Netmiko is that the overall usage is common across vendors. Only the commands used are specific to each platform.



To install Netmiko, you can use pip3:

```
$ pip3 install netmiko
$ pip3 list | grep netmiko
netmiko                                3.4.0
```

The first thing you need to do is import the proper Netmiko device object. This object handles the SSH connection setup, teardown, and the sending of commands to the device. You used a similar approach with ncclient:

```
>>> from netmiko import ConnectHandler
```

You're now ready to establish an SSH connection to the network device and create a Netmiko device object. The `ConnectHandler` object handles the SSH connection to the network device:

```
>>> device = ConnectHandler(
...     host='nxos-spine1',
...     username='admin',
...     password='admin',
...     device_type='cisco_nxos'
... )
```

At this point, there is an active SSH connection from Python using Netmiko with a Cisco NX-OS switch. Because each platform supports different commands and handles SSH differently, you must provide the `device_type` parameter when instantiating an instance of the `ConnectHandler` object.

Let's check the available methods for our new device object called `device` by using the `dir()` function:

```
>>> dir(device)
[
    # methods removed for brevity
    'cleanup', 'clear_buffer', 'close_session_log', 'commit', 'config_mode',
    'conn_timeout', 'device_type', 'disable_paging', 'disconnect', 'enable',
```

```

    'encoding', 'establish_connection', 'exit_config_mode', 'exit_enable_mode',
    'select_delay_factor', 'send_command', 'send_command_expect',
    'send_command_timing', 'send_config_from_file', 'send_config_set',
]

```

As a network engineer, you should feel pretty comfortable with many of the attributes shown from the `dir()` function, as they are very network centric. We'll walk through a few of them now.

## Verifying the device prompt

Use the `find_prompt()` method to check the prompt string of the device:

```

>>> device.find_prompt()
'nxos-spine1#'

```

## Entering configuration mode

Because Netmiko understands multiple vendors and what configuration mode means, it has a method to go into configuration mode that works across vendors; of course, the commands Netmiko uses under the covers may be different per OS:

```

>>> device.config_mode()
>>>
>>> device.find_prompt()
'nxos-spine1(config)#'

```



Some NOSs could fail to enter `config_mode()` if there is already a CLI session in this mode.

## Sending commands

The most common operation you're going to perform with Netmiko is sending commands to a device. Let's look at a few methods to do this.

To simply send a single command to a device, you can use one of three methods:

### `send_command_expect()`

This method is used for long-running commands that may take a while for the device to process (`show run` on a larger chassis, `show tech`, etc.). By default, this method waits for the same prompt string to return before completing. Optionally, you can pass what the new prompt string is going to be should it change based on the commands being sent.

### `send_command_timing()`

This method is for short-running commands; it is timing based and does not check the prompt string.

`send_command()`

This is an older method in Netmiko, which now acts as a wrapper for calling `send_command_expect()`. Thus, `send_command()` and `send_command_expect()` perform the same operation.

Let's look at a few examples. Here you're gathering a `show run` and printing out the first 176 characters for verification:

```
>>> show_run_output = device.send_command('show run')
>>>
>>> print(show_run_output[:176])

!Command: show running-config
!Running configuration last done at: Wed Oct  5 04:18:12 2022
!Time: Wed Oct  5 04:23:22 2022

version 9.3(3) Bios:version
hostname nxos-spine1
```

Send a command that changes the prompt string—remember you're still in configuration mode when you enter `device.config_mode()`—as follows:

```
>>> output = device.send_command_expect('end')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/usr/local/lib/python3.8/site-packages/netmiko/base_connection.py",
line 1582, in send_command_expect
    return self.send_command(*args, **kwargs)
  File "/usr/local/lib/python3.8/site-packages/netmiko/utilities.py", line 500,
in wrapper_decorator
    return func(self, *args, **kwargs)
  File "/usr/local/lib/python3.8/site-packages/netmiko/base_connection.py",
line 1535, in send_command
    raise IOError()
OSError: Search pattern never detected in send_command: nxos\-spine1(config)\#
>>>
```

The stack trace shown is expected, as `send_command_expect()` expects to see the same prompt string by default. Since you are in config mode with the current prompt string of `nxos-spine1(config)#`, when you type the command `end`, the new prompt string is going to be `nxos-spine1#`.

To execute a command that changes the prompt string, you have two options. First, you can use the `expect_string` parameter that defines the new and expected prompt string:

```
>>> output = device.send_command_expect('end', expect_string='nxos-spine1#')
>>>
```

Second, you can use the `send_command_timing()` method, which is timing based and doesn't expect a particular prompt string to be found again:

```
>>> output = device.send_command_timing('end')
>>>
```

You’ve shown three methods thus far on how to send commands with Netmiko. Let’s look at two more useful ones, as you may want to send several commands at once instead of one at a time.

Netmiko also supports a method called `send_config_set()` that takes a parameter that must be iterable. We’ll show this using a Python list, but you can also use a Python set:

```
>>> commands = [
    'interface Ethernet1/1',
    'description configured by netmiko',
    'shutdown'
]
>>>
>>> output = device.send_config_set(config_commands=commands)
>>>
>>> print(output)
nxos-spine1(config)# interface Ethernet1/1
nxos-spine1(config-if)# description configured by netmiko
nxos-spine1(config-if)# shutdown
nxos-spine1(config-if)# end
nxos-spine1#
```

This method checks whether you’re already in configuration mode. If you aren’t, it goes into config mode, executes the commands, and by default, exits configuration mode. You can verify this by viewing the returned output, as shown in the previous example.

Finally, Netmiko has a method that can execute commands from a file. This allows you to do something like create a Jinja template, render it with variable data, write the data to a file, and then execute those commands from the file with the Netmiko method `send_config_from_file()`. Building on what we covered in Chapters 6 and 9, let’s see how to perform this workflow in [Example 10-38](#).

*Example 10-38. Sending commands from a file with Netmiko*

```
from netmiko import ConnectHandler
from jinja2 import Environment, FileSystemLoader

device = ConnectHandler(
    ...
)

interface_dict = {
    "name": "Ethernet1/2",
    "description": "Server Port",
    "vlan": 10,
    "uplink": False
}
```

```

# Create the custom commands, combining the Jinja config.j2 template
# with the data defined in interface_dict
ENV = Environment(loader=FileSystemLoader('.'))
template = ENV.get_template("config.j2")
commands = template.render(interface=interface_dict)

# Store the CLI commands in a local file
filename = 'nxos.conf'
with open(filename, 'w') as config_file:
    config_file.writelines(commands)

# Send CLI commands directly from the file
output = device.send_config_from_file(filename)

# Use show commands to verify that the change succeeded
verification = device.send_command(f'show run interface {interface_dict["name"]}')
print(verification)

device.disconnect()

```

Everything shown in this example was covered in prior chapters. Note that *config.j2* must be created for this to work, and for this example, that the Jinja template is stored in the same directory from where we entered the Python interpreter. The content of the template is from [Example 9-3](#), and is as follows:

```

interface {{ interface.name }}
description {{ interface.description }}
switchport access vlan {{ interface.vlan }}
switchport mode access

```

Finally, when you're done working with Netmiko, you can gracefully disconnect from the device by using the `disconnect()` method. If we run the script, we will see the verification of the new interface configuration according to the template:

```

ch10-apis/python_netmiko$ python3 send_commands_from_file.py

!Command: show running-config interface Ethernet1/2
!Running configuration last done at: Wed Oct  5 04:38:54 2022
!Time: Wed Oct  5 04:38:55 2022

version 9.3(3) Bios:version

interface Ethernet1/2
description Server Port
switchport access vlan 10

```



Context managers in Python help manage setup and teardown operations. For an SSH library such as Netmiko, a context manager seems ideal. Thus, Netmiko provides one, `netmiko.ConnectHandler`, that will take care of establishing the SSH session at the beginning, and tearing it down when exiting it (so you don't leave open SSH connections):

```
with netmiko.ConnectHandler(**device_config) as device:
    device.send_command("show run")
```

So far, you have shown the benefits of Netmiko, allowing interaction with a traditional CLI interface programmatically. Unfortunately, the unstructured data used in the CLI output is a big drawback in the automation journey. Hopefully, we have some helpers available.

## Empowering Netmiko with TextFSM and NTC Templates

**TextFSM** is an open source project built by Google that converts semiforformatted text (the CLI output) to structured data, using templates. So, for each CLI output, you need to provide a specific template that NTC Templates solves.

**NTC Templates** is an open source project sponsored by Network to Code that provides a large collection of TextFSM templates for a lot of **network vendors**.



You don't need to install TextFSM or NTC Templates because they are dependencies of Netmiko, so they are already installed.

In **Example 10-39**, we demonstrate how to use NTC Templates in two steps:

1. Get raw CLI output from Netmiko and store it as a string.
2. Use the NTC Templates parser to transform the raw output into structured data.

*Example 10-39. Using NTC Templates to get structured data from Netmiko output*

```
>>> from netmiko import ConnectHandler
>>> device = ConnectHandler(
...     host='nxos-spine1',
...     username='admin',
...     password='admin',
...     device_type='cisco_nxos'
... )
>>> show_interfaces_raw = device.send_command('show int brief')
>>> show_interfaces_raw[:150]
'\n-----\n'
```

```

Port    VRF          Status IP Address      S'
>>>
>>> from ntc_templates.parse import parse_output
>>> show_interfaces_parsed = parse_output(
...     platform="cisco_nxos", ❶
...     command="show int brief", ❷
...     data=show_interfaces_raw, ❸
... )
>>> show_interfaces_parsed[0]
{
  'interface': 'mgmt0', 'vrf': '--', 'status': 'up', 'ip': '10.0.0.15',
  'speed': '1000', 'mtu': '1500', 'vlan': '', 'type': '', 'mode': '',
  'reason': '', 'portch': '', 'description': ''
}

```

- ❶ Indicates the reference platform. Each will have different parsers.
- ❷ Identifies the specific template within a platform because each CLI command may have different data.
- ❸ The raw input data to be parsed.

Luckily, since its 2.0.0 release, Netmiko has the implicit support of NTC Templates, simply using the `use_textfsm` argument:

```

>>> show_interfaces_parsed_directly = device.send_command(
...     'show int brief',
...     use_textfsm=True,
... )
>>> show_interfaces_parsed == show_interfaces_parsed_directly
True

```

This functionality is just combining the two steps presented in [Example 10-39](#).



Netmiko is also used as the primary SSH driver for devices within NAPALM, a robust and multivendor network Python library for configuring devices and retrieving data. We cover NAPALM in [Chapter 12](#).

This concludes using Netmiko to automate SSH-based network devices. You’ve now seen how to automate various types of network devices across a range of API types, no matter the device or API type you need to work with.